

Diagramme de classes — Framework Connector (business/payment)

Audience : architectes, tech leads, contributeurs au kernel, auteurs de connecteurs tiers

Périmètre : toutes les classes Java du framework connector business/payment, à travers les cinq modules qui les portent — `payos-connector-api` , `payos-connector-sdk` , `payos-foundation` , `payos` (kernel), `idempotency-service-redis`

Created : 2026-09-02

Dernière mise à jour : 2026-09-02

Version : v1

Table des matières

1. [Objectif et méthodologie](#)
2. [Légende](#)
3. [Vue d'ensemble — les classes pivots](#)
4. [payos-connector-api — le contrat partagé](#)
5. [payos-connector-sdk — le kit auteur de connecteurs](#)
6. [payos-foundation — le SPI côté kernel](#)
7. [payos \(kernel\) — boot, scan des JARs, cycle de vie](#)
8. [payos \(kernel\) — registre tenant, résolution, santé](#)
9. [payos \(kernel\) — hot reload, arrêt, réglages runtime](#)
10. [payos \(kernel\) — politiques retry / dédoublonnage / routage terminal](#)
11. [payos \(kernel\) — intégration scripting et points d'entrée](#)
12. [payos \(kernel\) — implémentations état & idempotence, diagnostics](#)
13. [idempotency-service-redis — backend Redis](#)
14. [Un piège de nommage à connaître](#)
15. [Références](#)

1. Objectif et méthodologie

Ce document est le complément au niveau classe de [connector-framework-architecture-v1-2026-08-24.md](#), qui décrit l'architecture en couches et les flux d'exécution du framework connector business/payment (`IConnector` , `$Connector(...)` , JAR tiers chargés à chaud). Là où ce document narratif explique *pourquoi* le framework est structuré en quatre modules, celui-ci répond à une question différente — *quelles classes existent, où vivent-elles, et comment se relient-elles* — en couvrant l'intégralité des classes/interfaces/enums/records du framework, à travers les cinq modules qui les portent.

Chaque classe listée ici a été localisée par lecture directe du code source (pas d'inférence à partir de noms) dans les répertoires suivants :

`payos-connector-api/src/main/java/ma/s2m/payos/connector/` , `payos-connector-sdk/src/main/java/ma/s2m/payos/connector/` , `payos-foundation/src/main/java/ma/s2m/payos/{config/connector,connector/state,idempotency}/` , `payos/src/main/java/ma/s2m/payos/{config/connector,connector,scripting,idempotency}/` , `idempotency-service-redis/src/main/java/ma/s2m/payos/idempotency/redis/` . Les classes de test (`*Test.java`) sont exclues du périmètre — ce document couvre uniquement le code de production. `ApiResourceHandler` , `HookEngine` et `BootServer` apparaissent en §11 comme points d'entrée de contexte (ils appellent le framework) mais ne sont pas eux-mêmes des classes du framework connector — leurs membres ne sont donc pas détaillés.

Le framework compte **106 classes/interfaces/enums/records** de production (plus 3 types imbriqués privés — `TenantConnectorRegistry.Builder` , `RecordDto` / `ConnectorResponseDto` de `RedisConnectorIdempotencyStore` , `CliArguments` de `ConnectorCertificationCli`). Un unique diagramme mermaid les représentant toutes serait illisible ; ce document les répartit donc en dix diagrammes ciblés par module et par responsabilité (§4 à §13), plus une vue d'ensemble (§3) qui relie les classes pivots de chaque couche. Ensemble, ces dix diagrammes constituent le diagramme de classes complet — chaque classe du périmètre apparaît dans au moins un diagramme, avec ses relations d'héritage (`--|>`), d'implémentation (`..|>`), d'utilisation/dépendance (`..>`) et de composition/agrégation (`*-- / o--`) réelles, telles que lues dans le code.

Une classe référencée depuis un autre module que celui du diagramme courant est redéclarée en version « allégée » (nom + stéréotype d'origine, sans reprendre tous ses membres, déjà détaillés dans son propre diagramme) — c'est la seule façon de représenter des arêtes inter-modules dans des diagrammes mermaid qui sont sinon indépendants les uns des autres.

2. Légende

Chaque classe porte un stéréotype de nature (<<interface>> , <<abstract>> , <<enumeration>> , <<record>> , <<exception>> , aucun pour une classe concrète ordinaire) et un stéréotype de module d'origine, pour repérer immédiatement les arêtes qui traversent une frontière Maven :

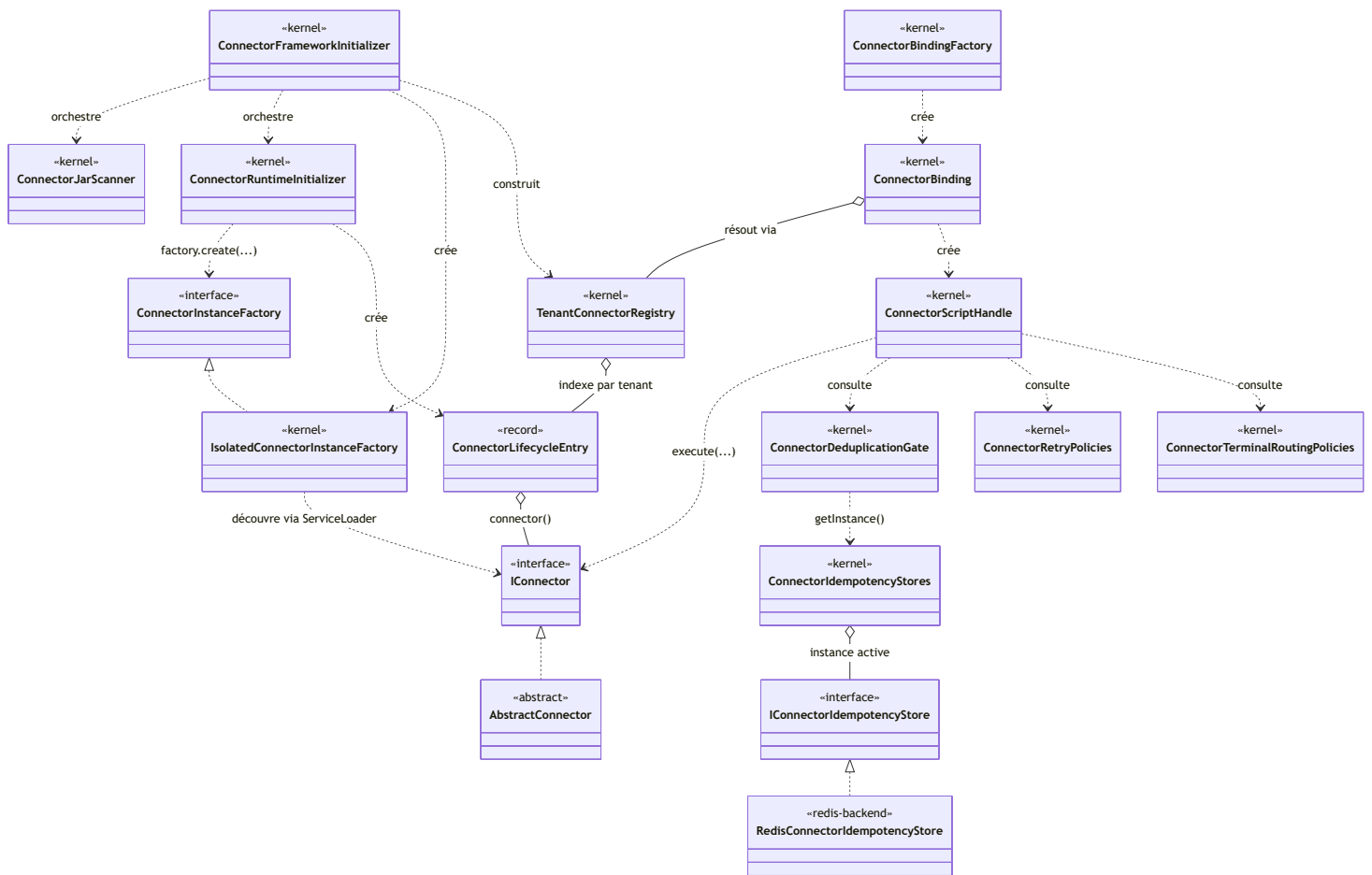
Stéréotype de module	Repository
<<connector-api>>	payos-connector-api
<<connector-sdk>>	payos-connector-sdk
<<foundation>>	payos-foundation
<<kernel>>	payos
<<redis-backend>>	idempotency-service-redis

Relations utilisées :

Notation mermaid	Signification
`A --`	> B`
`A ..`	> B`
A ..> B : label	A utilise/instancie/dépend de B (appel, new B(...) , paramètre, valeur de retour)
A *-- B	A possède B en composition (champ non partageable, souvent un composant de record)
A o-- B	A possède B en agrégation (référence à un objet qui existe indépendamment)

3. Vue d'ensemble — les classes pivots

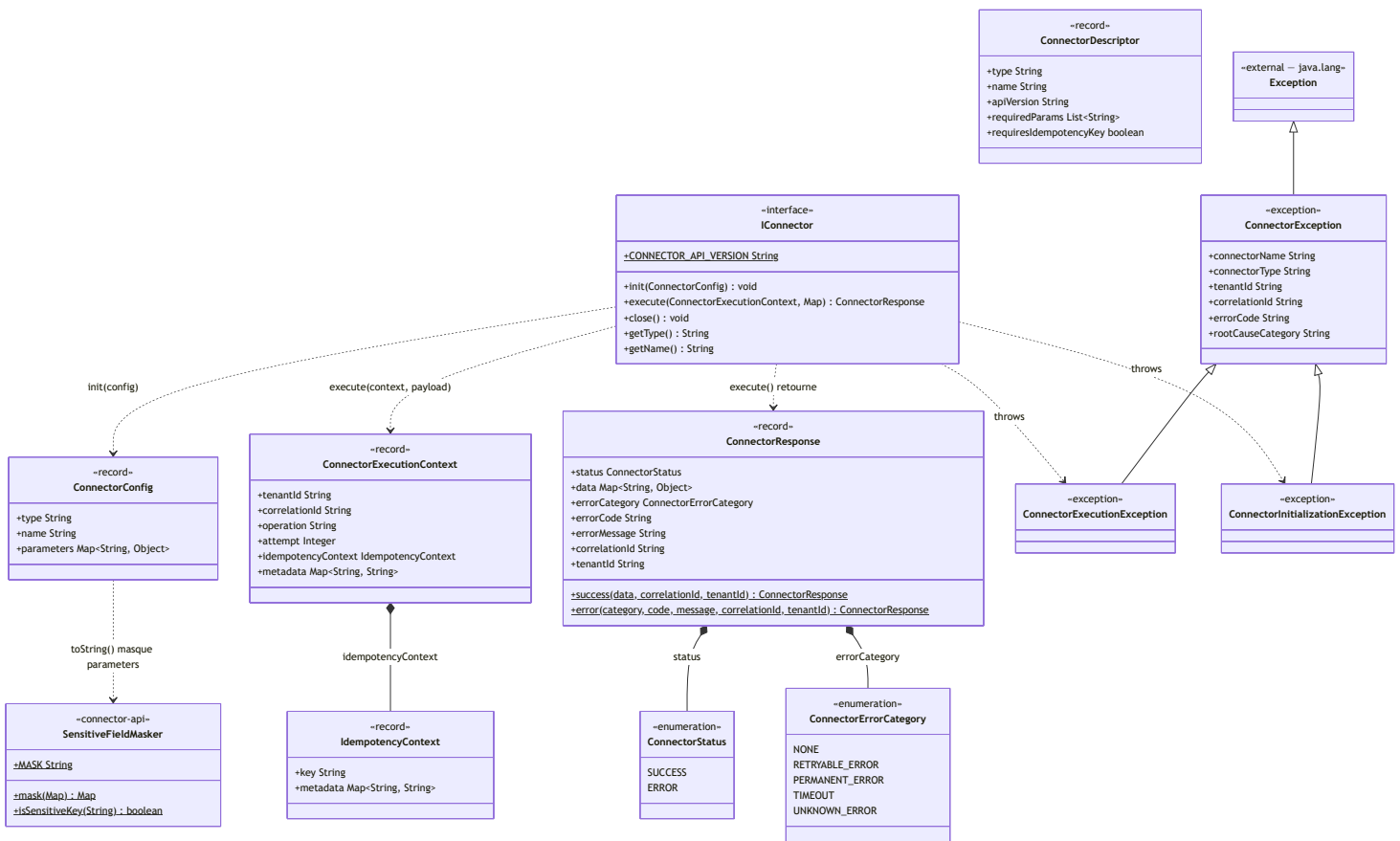
Ce diagramme relie les classes qui forment l'épine dorsale du framework à travers les cinq modules — le détail complet de chacune est dans les sections suivantes.



Un connecteur tiers implémente `IConnector` (directement ou, presque toujours, via `AbstractConnector`) ; le kernel le charge, le suit dans un `ConnectorLifecycleEntry`, l'indexe dans un `TenantConnectorRegistry` ; un script y accède via `$Connector(...)` (`ConnectorBindingFactory` → `ConnectorBinding` → `ConnectorScriptHandle`) qui invoque `IConnector.execute(...)` après consultation des politiques de dédoublement/retry/routage, elles-mêmes adossées à un `IConnectorIdempotencyStore` — implémenté en mémoire dans le kernel ou par `RedisConnectorIdempotencyStore` dans `idempotency-service-redis`.

4. payos-connector-api — le contrat partagé

Module feuille (12 classes) : zéro dépendance PayOS, seulement `java.util.*`. C'est le vocabulaire que tous les autres modules partagent.

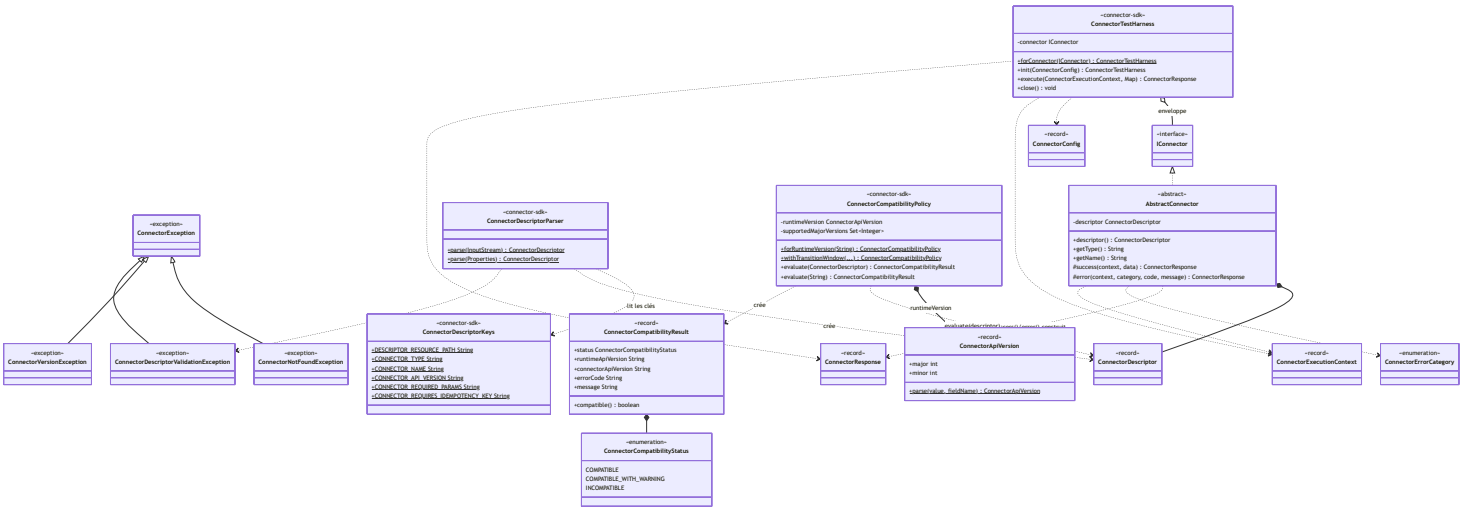


ConnectorConfig , ConnectorExecutionContext , ConnectorResponse , IdempotencyContext , ConnectorDescriptor sont des Java records avec validation dans le constructeur compact (non représentée ici — voir le code source pour le détail des contraintes requireNotBlank /normalisation).

ConnectorException porte l'identité complète d'une erreur connecteur (connectorName , connectorType , tenantId , correlationId , errorCode , rootCauseCategory) — ses deux sous-classes n'ajoutent aucun champ, seulement une sémantique (levée par init(...) vs execute(...)).

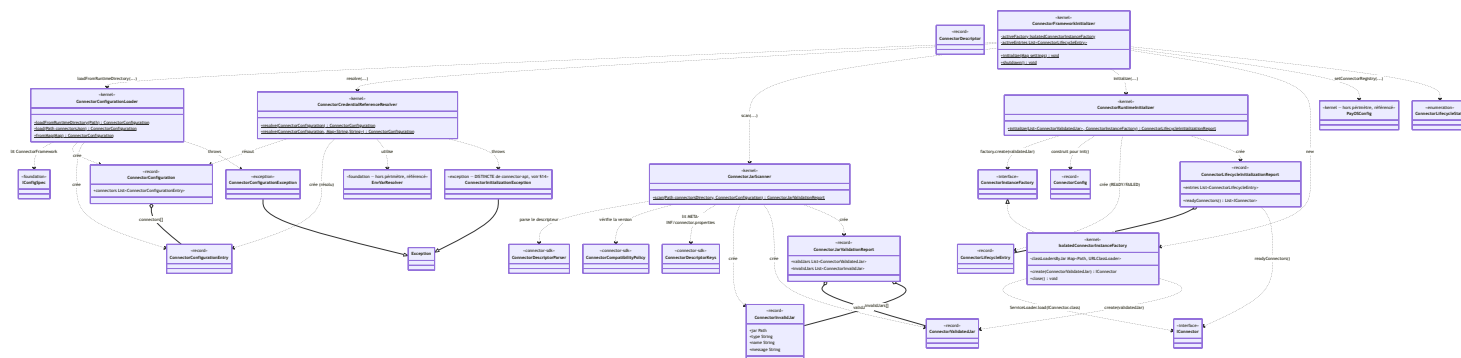
5. payos-connector-sdk — le kit auteur de connecteurs

24 classes, quatre sous-domaines : le point de départ pour écrire un connecteur (spi , test), le parsing du descripteur (descriptor), la compatibilité de version (compatibility), et la certification avant livraison (certification , certification.approval). Dépend uniquement de payos-connector-api .



7. payos (kernel) — boot, scan des JARs, cycle de vie

`ConnectorFrameworkInitializer` est la racine de composition du framework — l'orchestrateur que `BootServer` appelle au démarrage et à chaque hot reload. Ce diagramme couvre 12 classes du package `ma.s2m.payos.config.connector`.

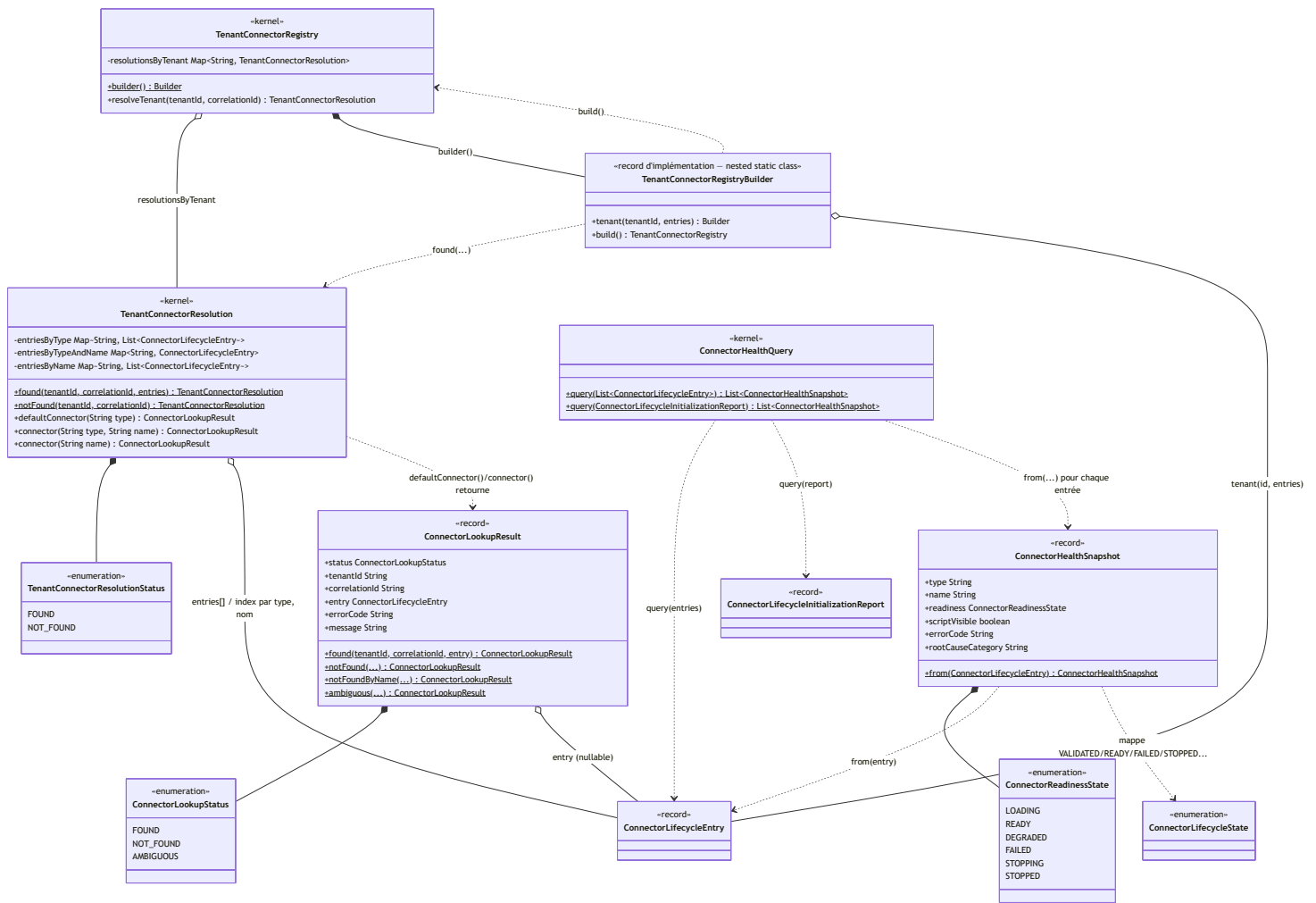


Le flux est linéaire et correspond exactement au diagramme de séquence §6.1 de [connector-framework-architecture-v1-2026-08-24.md](#) :

ConnectorConfigurationLoader (shape statique de connectors.json) → ConnectorCredentialReferenceResolver (résolution `{...}`) → ConnectorJarScanner (validation par JAR, délègue au SDK pour le parsing/la compatibilité) → ConnectorRuntimeInitializer (instanciation via IsolatedConnectorInstanceFactory, qui implémente l'interface ConnectorInstanceFactory déclarée dans payos-foundation) → construction du TenantConnectorRegistry (§8).

8. payos (kernel) — registre tenant, résolution, santé

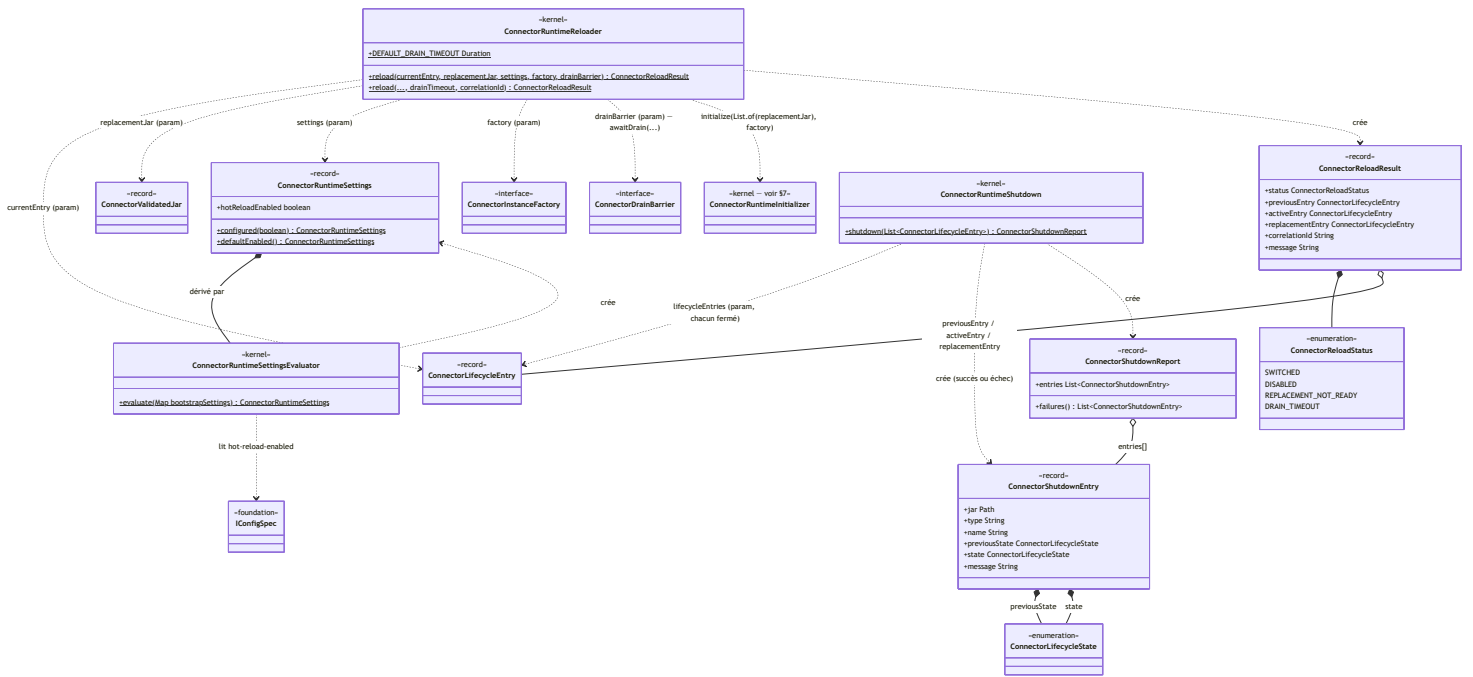
8 classes qui indexent les `ConnectorLifecycleEntry` prêts par tenant et exposent la résolution utilisée par `$connector(...)` ainsi qu'une vue de santé non filtrée par tenant.



TenantConnectorRegistry est immuable et construit une fois par ConnectorFrameworkInitializer (§7) — son Builder (classe statique imbriquée, notée TenantConnectorRegistryBuilder ici pour éviter une collision de nom dans le diagramme) rejette les doublons type+nom par tenant. Aujourd'hui tous les tenants voient la même liste de connecteurs (architecture §6.1), donc tenant(...) est appelé soit une fois avec "default", soit une fois par tenant déclaré avec la même liste. ConnectorHealthQuery / ConnectorHealthSnapshot sont indépendants du registre tenant — ils exposent une vue de diagnostic non filtrée, utile pour l'observabilité opérationnelle.

9. payos (kernel) — hot reload, arrêt, réglages runtime

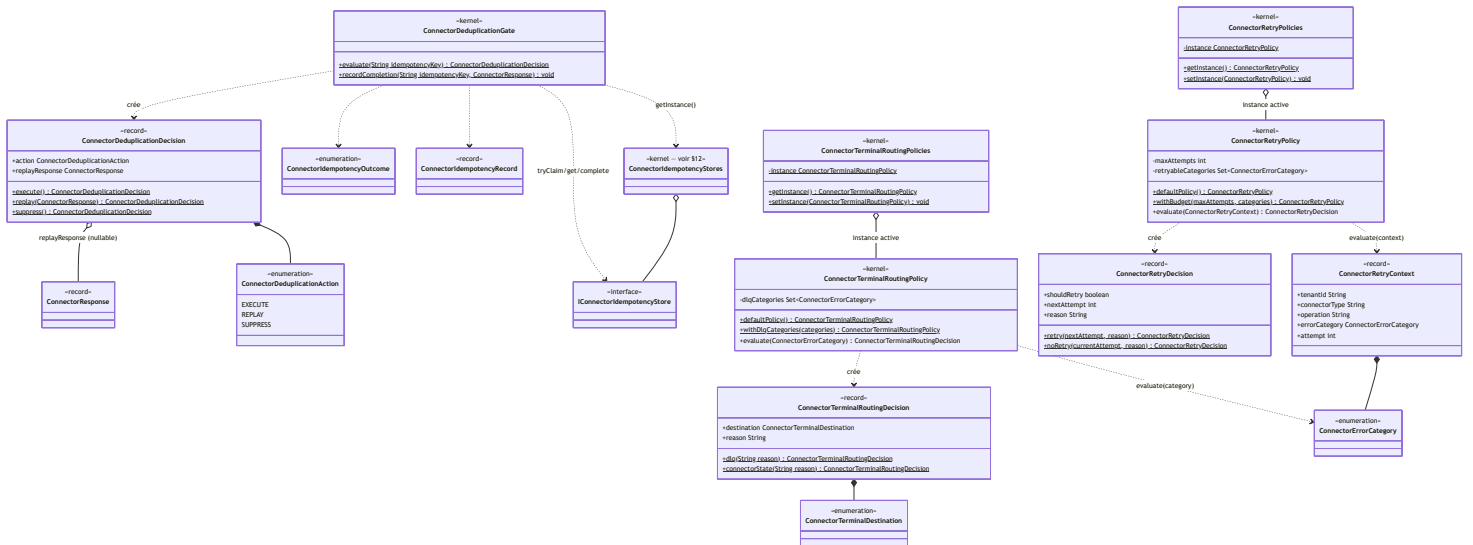
8 classes qui implémentent le cycle §6.4/§6.5 de l'architecture narrative — remplacement à chaud d'un connecteur avec drain, et fermeture ordonnée à l'arrêt.



`ConnectorRuntimeReloader.reload(...)` réutilise directement `ConnectorRuntimeInitializer.initialize(...)` (§7) pour préparer le remplacement — c'est la même mécanique d'instanciation que celle du boot, appliquée à un seul JAR. `ConnectorDrainBarrier` et `ConnectorInstanceFactory` sont deux interfaces `@FunctionalInterface` de `payos-foundation` reçues en paramètre — le kernel fournit `IsolatedConnectorInstanceFactory` comme implémentation de la seconde, mais aucune classe de production n'implémente `ConnectorDrainBarrier` dans ce périmètre (il est fourni par l'appelant, typiquement câblé côté tests ou par une future intégration opérationnelle).

10. payos (kernel) — politiques retry / dédoublonnage / routage terminal

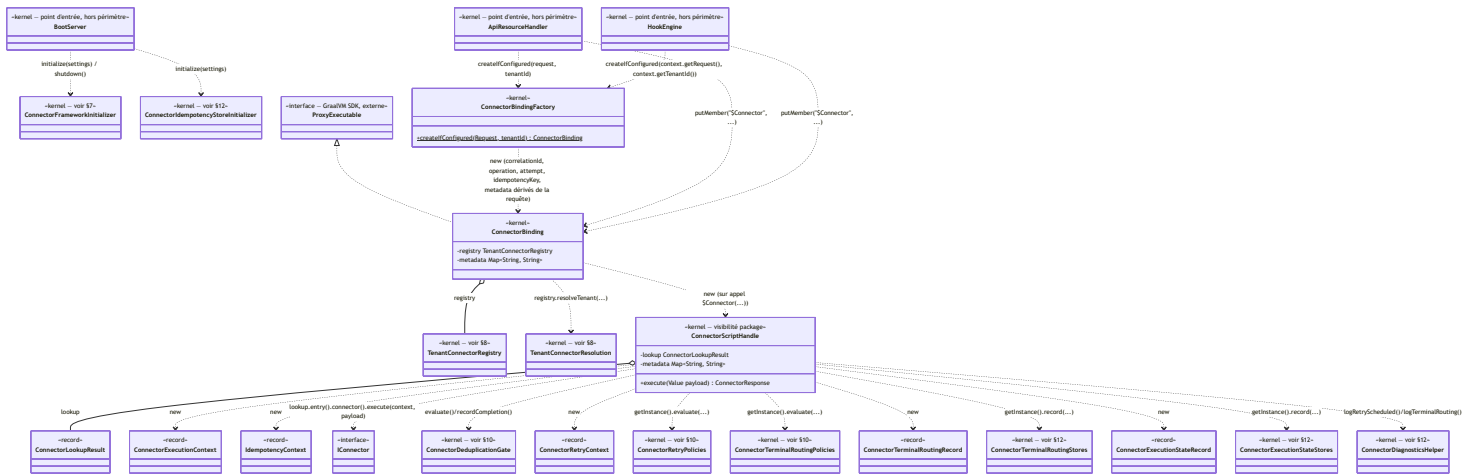
10 classes qui incarnent la règle centrale du framework : *la plateforme, jamais le connecteur, décide de la déduplication, du retry et du routage d'erreur*. Détail comportemental exhaustif dans [connector-framework-parameters-v3-2026-08-11.md §6–§11](#) — ce diagramme n'en montre que la structure de classes.



Ces trois politiques (`ConnectorDeduplicationGate`, `ConnectorRetryPolicy`, `ConnectorTerminalRoutingPolicy`) sont toutes consultées séquentiellement par `ConnectorScriptHandle.execute(...)` (§11), jamais par le connecteur lui-même. `ConnectorRetryPolicies` et `ConnectorTerminalRoutingPolicies` suivent le même patron que `ConnectorIdempotencyStores` / `ConnectorExecutionStateStores` / `ConnectorTerminalRoutingStores` (§12) : une façade statique swappable au-dessus d'une instance mutable, le même style de singleton configurable répété à travers tout le kernel connecteur.

11. payos (kernel) — intégration scripting et points d'entrée

Les 3 classes qui rendent `$Connector(...)` disponible depuis un script JS, plus les 3 points d'entrée kernel qui les câblent (montrés ici pour le lien d'usage demandé, sans détailler leurs membres — ils ne font pas partie du framework connecteur).

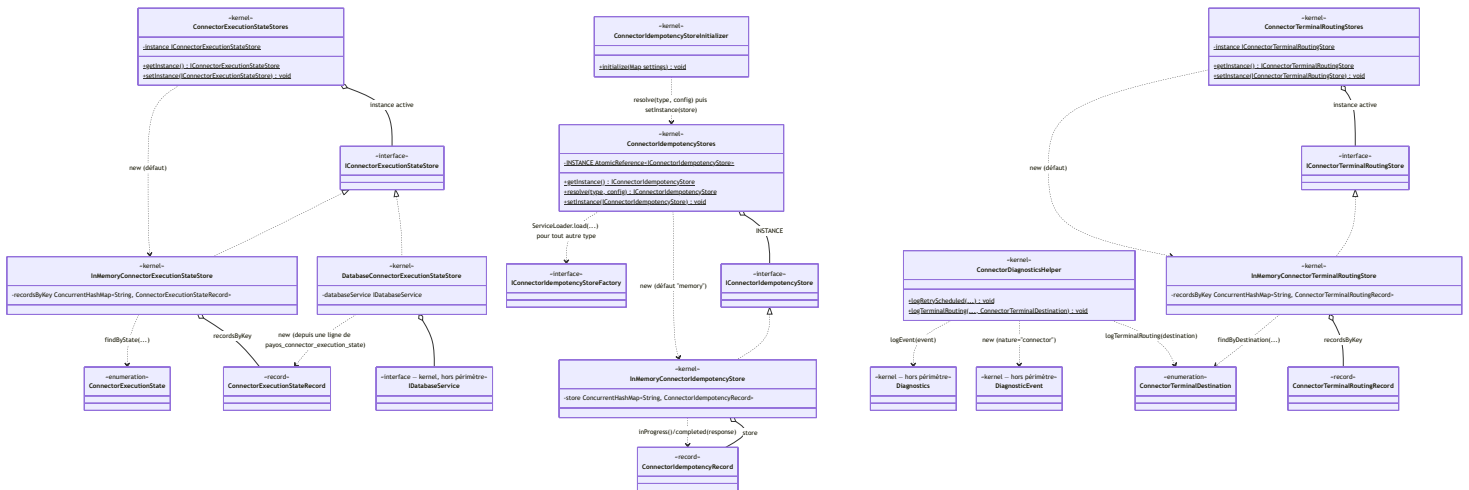


`ConnectorScriptHandle.execute(...)` est le point de passage unique de toute exécution — c'est la classe qui matérialise le diagramme de séquence §6.3 de l'architecture narrative (idempotence → dédoublonnage → invocation → classification d'erreur → retry → routage terminal → diagnostics).

`ConnectorBindingFactory` centralise la dérivation de `correlationId` / `operation` / `attempt` / `idempotencyKey` / `metadata` depuis la requête pour que `ApiResourceHandler` et `HookEngine` — les deux seuls points d'entrée qui exposent `$Connector` à un script — n'aient pas à dupliquer cette logique.

12. payos (kernel) — implémentations état & idempotence, diagnostics

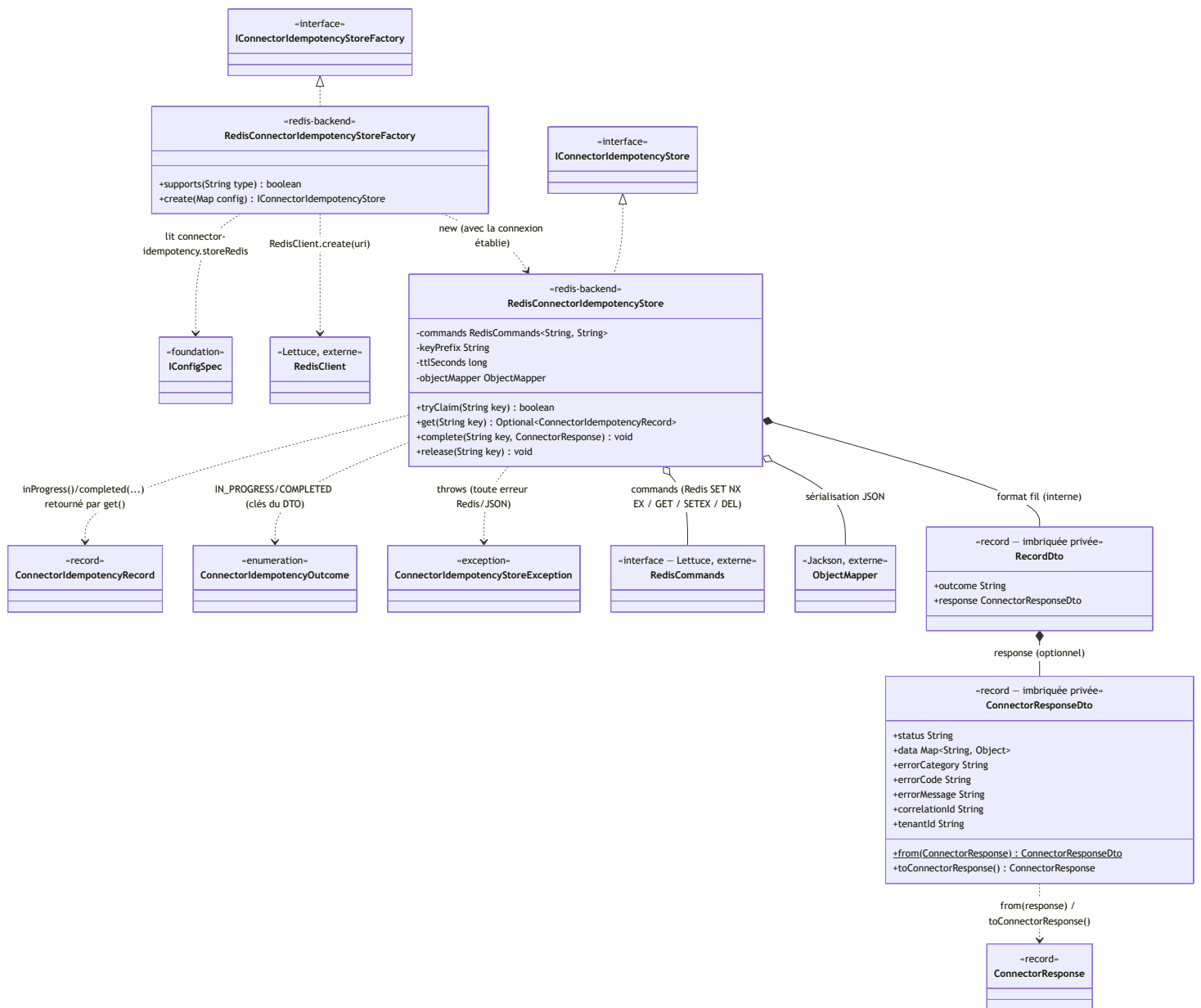
Les implémentations concrètes des interfaces `payos-foundation` de §6, plus l'aide au diagnostic. 9 classes.



Trois façades statiques swappables partagent le même patron : `ConnectorExecutionStateStores`, `ConnectorTerminalRoutingStores` et `ConnectorIdempotencyStores` démarrent chacune avec une implémentation en mémoire par défaut, remplaçable via `setInstance(...)`. Seul `ConnectorIdempotencyStores.resolve(...)` a un mécanisme de découverte automatique — `ServiceLoader.load(IConnectorIdempotencyStoreFactory.class)` — c'est ce mécanisme que `RedisConnectorIdempotencyStoreFactory` (§13) enregistre sous `META-INF/services`. `ConnectorExecutionStateStores` et `ConnectorTerminalRoutingStores` n'ont pas d'équivalent découverte-automatique aujourd'hui — passer `DatabaseConnectorExecutionStateStore` en instance active est un choix de câblage laissé à l'appelant, non automatisé par `BootServer`.

13. idempotency-service-redis — backend Redis

2 classes de production (+ 2 records DTO privés imbriqués) qui fournissent l'unique implémentation Redis du SPI `IConnectorIdempotencyStore` de `payos-foundation`.



`tryClaim(key)` s'appuie sur un `SET key ... NX EX ttlSeconds` Redis atomique — c'est la garantie native `NX` de Redis, pas un script Lua, qui empêche deux exécutions concurrentes sur la même clé d'idempotence de gagner toutes les deux la réclamation. `RedisConnectorIdempotencyStoreFactory` est découverte par `ConnectorIdempotencyStores.resolve("redis", config)` (§12) via `ServiceLoader` — le mécanisme d'extension standard déjà utilisé par `IdempotencyStoreFactory` (le store HTTP, hors périmètre de ce document).

14. Un piège de nommage à connaître

Deux paires de classes portent le **même nom simple** dans des packages différents, avec des hiérarchies totalement indépendantes — un risque de confusion réel si on ne regarde que le nom :

Nom simple	Occurrence 1	Occurrence 2
<code>ConnectorInitializationException</code>	<code>ma.s2m.payos.connector.exception.ConnectorInitializationException</code> (<code>payos-connector-api</code>), étend <code>ConnectorException</code> — levée par <code>IConnector.init(...)</code>	<code>ma.s2m.payos.config.connector.Connector</code> (<code>payos kernel</code>), étend <code>java.lang.Exception</code> par <code>ConnectorCredentialReferenceResolve</code> credential non résolue

Aucune relation d'héritage ne relie ces deux classes malgré le nom identique — un import erroné (`connector.exception` au lieu de `config.connector`, ou l'inverse) compile souvent quand même si le code environnant catch `Exception`, ce qui peut masquer une confusion de type au code review. Vérifier

systématiquement le package complet dans les imports plutôt que le nom simple de la classe.

15. Références

- [connector-framework-architecture-v1-2026-08-24.md](#) — l'architecture narrative en couches (le document qui explique le *pourquoi* ; celui-ci détaille le *quoi*).
- [connector-framework-parameters-v3-2026-08-11.md](#) — référence exhaustive des paramètres de configuration et des règles de comportement (catégories d'erreur, retry, routing terminal, diagnostics) derrière les classes du §10.
- [extensibility.md](#) — vue d'ensemble des neuf mécanismes d'extension de PayOS.
- Code source : `payos-connector-api/src/main/java/ma/s2m/payos/connector/` ; `payos-connector-sdk/src/main/java/ma/s2m/payos/connector/` ; `payos-foundation/src/main/java/ma/s2m/payos/{config/connector,connector/state,idempotency}/` ; `payos/src/main/java/ma/s2m/payos/{config/connector,connector,scripting,idempotency}/` ; `idempotency-service-redis/src/main/java/ma/s2m/payos/idempotency/redis/` .

Note de méthode : ce document a été produit par lecture exhaustive du code source de production (hors tests) le 2026-09-02. Toute classe ajoutée, renommée ou déplacée dans le framework connecteur après cette date n'y figure pas — recouper avec le code source pour un audit ultérieur plutôt que de considérer ce document comme une source de vérité indéfiniment à jour.